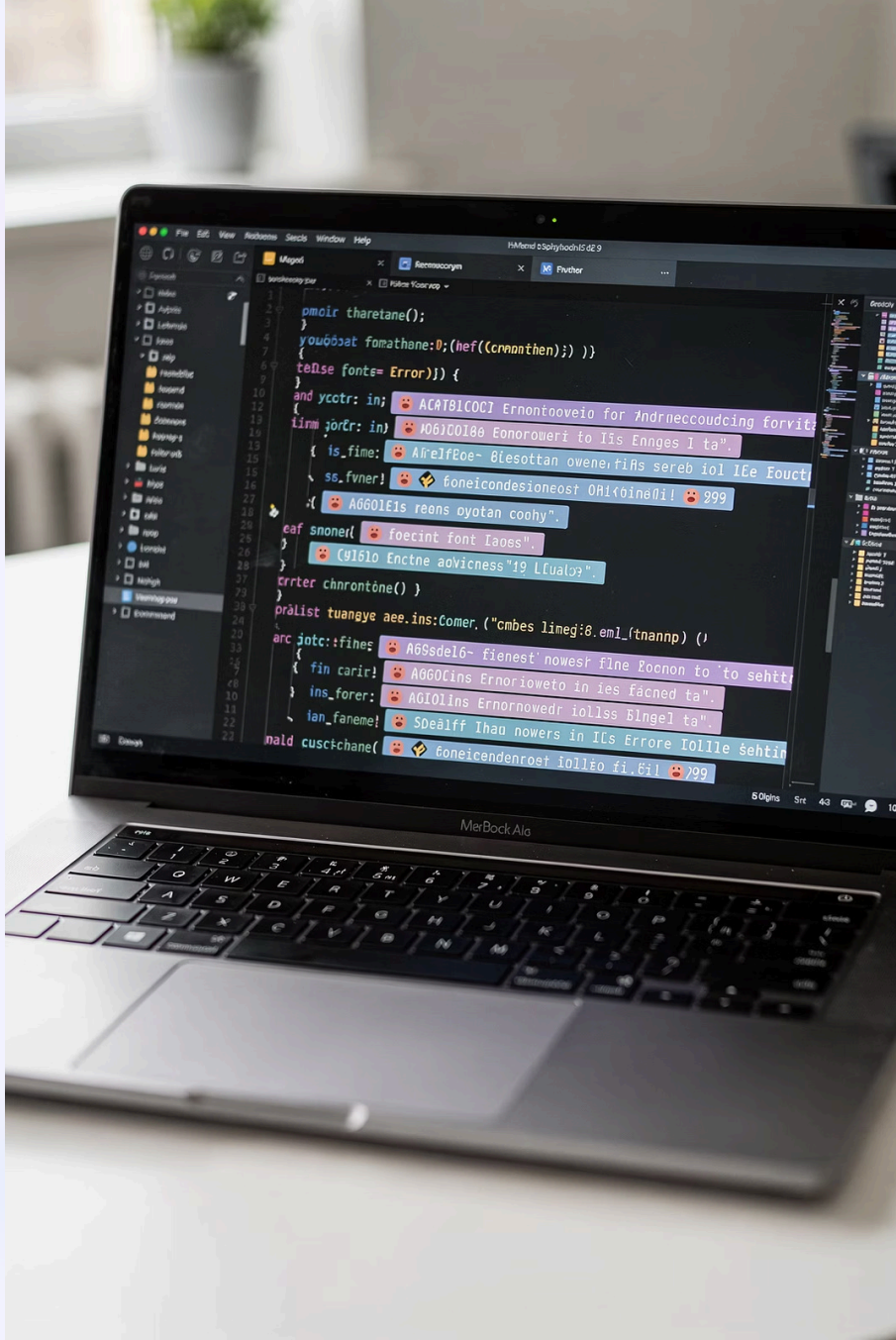
Manejo de Errores en Python

Aprende a identificar, gestionar y resolver errores para crear aplicaciones más robustas y estables



INTRODUCCIÓN

¿Por qué es importante manejar errores?

El desafío

Los errores son inevitables en la programación. Pueden surgir en cualquier momento durante el desarrollo o ejecución de un programa.

Como programadores, debemos estar preparados para anticipar y gestionar situaciones problemáticas. El manejo adecuado de errores no solo previene que nuestras aplicaciones se detengan inesperadamente, sino que también mejora la experiencia del usuario y facilita el mantenimiento del código.

En esta clase, exploraremos los tipos de errores más comunes en Python y aprenderemos técnicas profesionales para gestionarlos eficientemente.

OBJETIVOS

Resultados de Aprendizaje



Comprensión de Errores

Identificar y gestionar errores comunes: de sintaxis, en tiempo de ejecución y excepciones.



Uso de Excepciones

Aplicar bloques try-except para capturar errores y mantener la ejecución del programa.



Prevención y Resolución

Mejorar la robustez de tus programas gestionando condiciones excepcionales de manera eficiente.

CONTENIDO

Tipos de Errores que Exploraremos



1. **Syntax Error**

   Errores en la estructura del código que impiden su ejecución

2. **Name Error**

   Uso de variables o funciones no definidas

3. **Type Error**

   Operaciones con tipos de datos incompatibles

4. **Zero Division Error**

   Intentos de división por cero

5. **Index Error**

   Acceso a índices fuera del rango de una lista

TIPO 1

Syntax Error – Error Sintáctico

¿Qué es?

Los errores de sintaxis ocurren cuando el código no sigue las reglas gramaticales de Python. Son los más fáciles de detectar porque Python ni siquiera intentará ejecutar el programa.

Ejemplos comunes:

- Olvidar cerrar paréntesis o comillas
- Usar palabras reservadas incorrectamente
- Indentación incorrecta
- Dos puntos faltantes después de if, for, while

💡 Ejemplo práctico

```
# Código incorrecto con SyntaxError
if True
    print("¡Esto causará un error!")

# Python espera ':' después de la condición 'if'
# El error sería: SyntaxError: expected ':'
```

TIPO 2

Name Error – Error de Nombre

¿Qué es?

Este error aparece cuando intentamos usar una variable, función o módulo que no ha sido definido previamente. Python no puede encontrar el nombre al que estamos haciendo referencia.

Causas frecuentes:

- Escribir mal el nombre de una variable
- Usar una variable antes de asignarle un valor
- Olvidar importar un módulo
- Confundir mayúsculas y minúsculas

```
# Código incorrecto con NameError
print(variable_inexistente)

# Python no encuentra 'variable_inexistente'
# El error sería: NameError: name
'variable_inexistente' is not defined
```

🎯 Consejo

Python es **case-sensitive**. Las variables `edad`, `Edad` y `EDAD` son diferentes.

TIPO 3

Type Error – Error de Tipo

¿Qué es?

Los errores de tipo surgen cuando intentamos realizar operaciones con tipos de datos incompatibles. Por ejemplo, no podemos sumar directamente un número con un texto.

Operaciones inválidas

Intentar sumar un string con un entero

```
"5" + 3  # TypeError: can only concatenate str (not "int") to str
```



TIPO 4

Zero Division Error



¿Qué es?

Este error ocurre cuando intentamos dividir un número por cero, una operación matemáticamente indefinida que Python no puede procesar.

Situaciones comunes:

- Cálculos con denominadores variables
- Divisiones en bucles donde el divisor puede ser cero
- Operaciones con inputs del usuario sin validar

> *"La división por cero es una de las operaciones más comunes que debemos prevenir en cálculos matemáticos."*

```
10 / 0  # ZeroDivisionError: division by zero
```

TIPO 5

Index Error – Error de Índice

¿Qué es?

Este error aparece cuando intentamos acceder a una posición que no existe en una lista, tupla o string. Recordemos que los índices en Python comienzan en 0.



Índices fuera de rango

Intentar acceder a una posición mayor que la longitud de la lista menos uno.

```
lista = [1, 2, 3]
print(lista[5]) # IndexError: list index out of range
```



GESTIÓN AVANZADA

El Poder de las Excepciones

¿Y si pudieras evitar que tu programa se detenga?

Las excepciones son el mecanismo de Python para manejar errores de forma elegante. Con el bloque `try-except`, podemos anticipar problemas, capturarlos y decidir qué hacer sin que el programa se detenga abruptamente. Es como tener un plan B automático para cada situación problemática.

Este enfoque no solo mejora la experiencia del usuario, sino que también facilita la depuración y el mantenimiento del código a largo plazo.

TRY-EXCEPT

Estructura Básica de Try-Except

¿Cómo funciona?

El bloque `try-except` permite ejecutar código que puede fallar. Si ocurre un error dentro del bloque `try`, Python salta inmediatamente al bloque `except` correspondiente en lugar de detener el programa.



Ejemplo básico:

```
try:
    # Código que puede fallar
    numero = int(input("Ingresá un número: "))
    resultado = 100 / numero
    print(f"Resultado: {resultado}")

except ValueError:
    print("Error: Debes ingresar un número válido")

except ZeroDivisionError:
    print("Error: No se puede dividir por cero")
```

Esta estructura nos da control total sobre cómo responder a diferentes tipos de errores.

COMPONENTES

Componentes del Bloque Try-Except



try

Contiene el código que queremos ejecutar y que potencialmente puede generar errores.



except

Define qué hacer cuando ocurre un error específico. Podemos tener múltiples bloques except para diferentes tipos de errores.



else

Se ejecuta solo si NO ocurrió ningún error en el bloque try. Es opcional pero útil para separar la lógica exitosa.



finally

Se ejecuta SIEMPRE, haya o no error. Ideal para tareas de limpieza como cerrar archivos o conexiones.

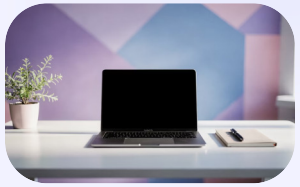
BUENAS PRÁCTICAS

Consejos para un Manejo Efectivo de Errores



Sé específico con las excepciones

Captura tipos de errores concretos en lugar de usar un `except` genérico. Esto facilita la depuración y el mantenimiento.



Mantén el bloque try pequeño

Solo incluye el código que realmente puede fallar. Esto hace más fácil identificar dónde ocurrió el problema.



Proporciona mensajes claros

Los mensajes de error deben ser informativos y orientar al usuario sobre qué salió mal y cómo solucionarlo.



Documenta tus excepciones

Explica por qué estás manejando ciertos errores y qué alternativas ofreces al usuario cuando ocurren.

RECURSOS

Materiales de Apoyo

Para profundizar tu comprensión sobre el manejo de errores en Python, te recomendamos estos recursos audiovisuales que complementan los contenidos de esta clase:

1

Tipos de Errores en Python

Video explicativo que recorre los cinco tipos de errores más comunes con ejemplos prácticos y casos de uso reales.

🔗 **Ver video en YouTube**

2

Excepciones en Python

Tutorial completo sobre cómo usar try-except para capturar y gestionar excepciones, mejorando la robustez de tus programas.

🔗 **Ver video en YouTube**

💡 **Tip:** Mirá los videos con un editor de código abierto para practicar los ejemplos en tiempo real. La práctica es clave para dominar el manejo de errores.



CONCLUSIÓN

¡Es tu turno de practicar!

Próximos pasos

01

Explora el notebook

Revisa todos los ejemplos prácticos disponibles

02

Mira los videos

Refuerza los conceptos con material audiovisual

03

Practica con ejercicios

Implementa try-except en tus propios programas

> **"Los errores son oportunidades para aprender. Un buen programador no es quien no comete errores, sino quien sabe manejarlos con elegancia."**

Ahora que conoces los tipos de errores más comunes y cómo gestionarlos con excepciones, estás listo para escribir código más robusto y profesional. ¡Adelante! 💪👨‍💻👩‍💻